技术项目说明

多智能体软件开发系统

基于 LangGraph 的五角色协作流水线与反馈循环机制

一、项目概述

本项目构建了一个基于 LangGraph 的多智能体软件开发系统，通过多个分工明确的智能体（Agent）协作完成软件开发任务。系统采用"产品经理 → 架构师 → 开发者 → 测试工程师 → 代码审查员"的流水线协作模式，并内置反馈循环机制：当测试失败或代码审查不通过时，任务自动回流到开发者节点进行修复，直至通过验收或达到迭代上限。

系统通过统一的 LLM 抽象层支持三种运行模式：模拟模型（MockLLM，无需任何外部依赖即可演示完整流程）、本地 Ollama 模型（默认 qwen2.5:3b，无需 API Key、数据不出本机）、OpenAI 兼容的真实大模型（可对接 DeepSeek、通义千问等）。系统提供 Web 界面，通过 SSE（Server-Sent Events）实时推送每个智能体的协作过程，实现"提交任务 → 多智能体协作 → 实时可视化 → 输出结果"的完整闭环。

二、技术方案

2.1 系统架构

系统采用分层架构，自底向上分为四层，各层职责如下表所示。

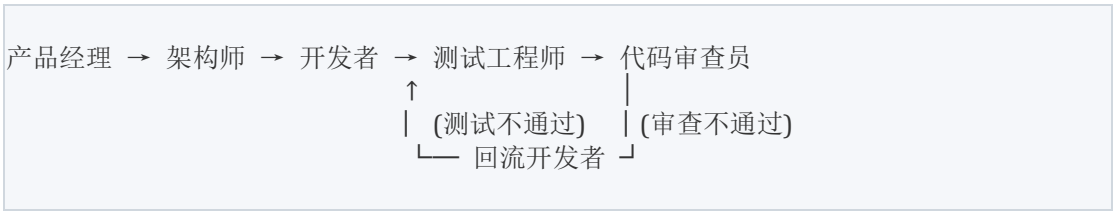
层次	模块	职责
LLM 抽象层	core/llm.py	统一 BaseLLM 接口，提供 MockLLM / OllamaLLM / OpenAILLM 三种实现，通过环境变量 LLM_PROVIDER 一键切换
Agent 角色层	agents/definitions.py	定义五个智能体角色的系统提示词、图标、颜色等元数据，形成角色注册表

层次	模块	职责
LangGraph 编排层	core/graph.py	使用状态图定义协作流程与条件路由，管理任务、历史、迭代次数等协作状态
Web 服务层	web/server.py + static/	Flask 提供 REST API 与 SSE 事件流，前端实现协作过程可视化

配置管理采用轻量级 .env 加载器（core/config.py），不引入 python-dotenv 等额外依赖；Ollama 接入直接调用其原生 /api/chat 接口，仅依赖 requests，无需安装 openai 包。

2.2 协作流程

五个智能体按流水线顺序执行，测试与审查环节设有条件回流，流程如下。



其中：产品经理分析需求、拆解任务、定义验收标准；架构师设计技术方案、划分模块、技术选型；开发者实现功能代码、编写注释、处理边界情况；测试工程师编写并运行单元测试、报告测试结果；代码审查员审查代码质量、发现缺陷、给出通过/不通过结论。

条件路由逻辑：测试与审查环节均要求模型以固定格式输出「结论： 通过/不通过」；路由层优先解析结构化结论标记，解析失败时退化为关键词匹配（含"不通过"等否定词优先判定，避免"不通过"被误判为"通过"）。

2.3 关键工程细节

迭代上限控制：开发者节点每次执行递增迭代计数，达到 MAX_ITERATIONS（默认 2）上限后通过 loop_capped 事件终止流程并通知前端，防止小模型输出不稳定导致的无限回流。

上下文预算控制：喂给每个智能体的前序产出限制在 MAX_CONTEXT_CHARS（默认 6000 字符）内，超出时保留最近的产出，防止超出 num_ctx 上下文窗口。

模型保活：本地 Ollama 请求携带 keep_alive 参数（默认 30 分钟），避免每次提交任务都重新加载模型（3B 模型冷启动约 20-30 秒）。

启动预检：服务启动时打印当前使用的 LLM 并执行 ensure_available()，Ollama 服务未启动或模型未下载时给出明确的修复提示。

三、创新点

角色化多智能体协作：将软件开发流程抽象为五个职责分明的智能体角色，每个角色拥有独立的系统提示词与职责边界，模拟真实团队协作。

内置反馈循环机制：不同于单向流水线，测试与审查环节设置条件路由，支持任务自动回流修复，并用结构化「结论」标记加关键词兜底的双重判定提升路由鲁棒性，配合迭代上限保证流程必然终止。

模拟模型与真实模型无缝切换：统一的 LLM 抽象层使系统可在零依赖的模拟模式下完整演示流程，也可切换到本地 Ollama 或云端大模型，兼顾演示、隐私与生产需求。

实时协作可视化：通过 SSE 事件流将 agent_start / agent_done / loop_capped 等事件实时推送到 Web 前端，直观呈现任务流转与各角色产出。

轻量可扩展：核心代码约 900 行，模块划分清晰，新增智能体角色只需在角色注册表中添加一条定义。

四、验证结果

系统已完成两条链路的端到端验证。

4.1 验证一：MockLLM 模拟链路

提交任务"开发一个待办事项管理应用"，验证流程正确性，结果如下。

验证项	结果
核心图执行顺序	pm → architect → developer → tester → reviewer，全部按序执行
协作历史记录	5 条（每个智能体各 1 条）
最终角色	reviewer（审查通过，流程正常结束）
REST API	/api/roles、/api/status、/api/run 均正常响应
SSE 事件流	正确推送 snapshot 及完整协作历史
运行耗时	约 2 秒（模板化输出，验证流程正确性）

4.2 验证二：本地 Ollama qwen2.5:3b 真实模型链路

在纯 CPU 环境（AMD64 笔记本，Windows 11）实测，结果如下。

验证项	结果
模型与接入方式	Ollama 原生 /api/chat，qwen2.5:3b（Q4_K_M，1.9 GB）
单次调用生成速度	约 7 tokens/s（纯 CPU 推理）
端到端任务	提交"写一个判断回文字符串的 Python 函数"，5 个智能体全部执行，约 342 秒完成
输出质量	开发者给出可用的 is_palindrome() 实现，测试工程师给出 unittest 用例，审查员输出结构化的优缺点分析
结论路由	测试与审查均按约定格式输出「结论： 通过」，条件路由正确终止流程
稳定性	另一任务（命令行待办工具）同样完整走通，约 386 秒

验证表明：系统在零依赖模拟模式与本地真实模型模式下均能完整执行多智能体协作流程，Web 界面与后端 API 工作正常，反馈循环与终止机制符合设计预期。

五、当前局限

小模型输出质量有限：qwen2.5:3b 在复杂需求下偶有答非所问或格式漂移，结论判定依赖提示词约束与关键词兜底，对自由文本的鲁棒性仍需加强。

纯 CPU 推理速度慢：约 7 tokens/s，一个完整任务约 5-7 分钟，交互体验受限。

反馈循环判定依赖文本标记：通过/不通过的判定基于「结论」标记与关键词匹配，真实模型自由输出时可能误判（已用迭代上限兜底，但会以"未收敛"告终）。

单任务串行执行：一次只能运行一个任务，不支持任务队列与并发。

无持久化存储：协作历史仅保存在内存中，服务重启后丢失。

前端功能较基础：缺少代码文件落地、测试报告下载等进阶功能。

六、后续计划

1. **接入更强模型：**配置 DeepSeek / 通义千问等云端 API 验证协作质量，并针对自由文本输出优化结论判定（如结构化 JSON 输出、LLM 自评）。
2. **增强反馈循环：**支持多轮迭代修复的精细控制、按角色定制迭代策略。
3. **支持多任务并发：**引入任务队列，支持批量提交与并行执行。
4. **持久化与历史管理：**接入数据库存储协作历史，支持任务回放与结果检索。
5. **代码落地与真实测试执行：**让开发者智能体真正生成可运行代码文件，测试智能体实际执行测试并生成报告。
6. **前端增强：**增加代码预览、测试报告下载、协作过程回放等交互功能。
7. **扩展智能体角色：**支持自定义角色（如安全审查员、性能优化师），提升系统通用性。